

BURGERS' EQUATION SOLVER

Finite Volume Method for Hyperbolic Equations (AS5460)

Jul-Nov 2026*

Kanak Agarwal[1]

[1] Department of Aerospace Engineering, Indian Institute of Technology, Madras, Chennai, India - 600036

* Submitted on September 26, 2026

Contents

1 Introduction **1**

2 Governing Equations and Numerical Method **1**

2.1 Godunov Scheme 1

2.2 Flux Vector Splitting (FVS) Scheme 2

2.3 Flux Difference Splitting (FDS) Scheme 2

3 Initialisation and Simulation Details **2**

4 Exact Solutions for the Inviscid Burgers Equation **3**

4.1 Case 1: Moving Shock 3

4.2 Case 2: Rarefaction 4

4.3 Case 3: Symmetric Rarefaction 4

4.4 Case 4: Stationary Shock 5

4.5 Case 5: Smooth Compression 5

5 Results and Discussion **6**

5.1 Case 1: Moving Shock 7

5.2 Case 2: Rarefaction 8

5.3 Case 3: Symmetric Rarefaction 9

5.4 Case 4: Stationary Shock 10

5.5 Case 5: Smooth Compression 11

6 Conclusion **12**

7 References **12**

A FORTRAN Implementation **12**

A.1 Variable Declaration Module 12

A.2 Main Program 13

A.3 Supporting Subroutines 15

B Python Post-Processing **18**

Figures

1 Initial Conditions 3

2 Solution corresponding to Case 1 7

3 Flux residuals corresponding to Case 1 7

4 Solution corresponding to Case 2 8

5 Flux residuals corresponding to Case 2 8

6 Solution corresponding to Case 3 9

7 Flux residuals corresponding to Case 3 9

8 Solution corresponding to Case 4 10

9 Flux residuals corresponding to Case 4 10

10 Solution corresponding to Case 5 11

11 Flux residuals corresponding to Case 5 11

1 Introduction

This report presents the results of the inviscid Burgers' equation solver developed as a part of the AS5460 course at the Indian Institute of Technology, Madras.

2 Governing Equations and Numerical Method

The governing equation is the inviscid Burgers' equation given by,

$$u_t + u\,u_x = 0 \tag{1}$$

Alternatively, this can be written in the conservation form as,

$$u_t + f_x = 0 \quad \forall \quad f = \frac{u^2}{2} \tag{2}$$

The governing equation (Eq. 1) is solved using the finite volume method (FVM) with explicit timestepping. Three flux schemes are adopted. These are the Godunov flux construction method, the flux vector splitting (FVS) and the flux difference splitting (FDS) methods. Mathematically, they can be represented as follows.

2.1 Godunov Scheme

In the Godunov scheme, the exact solution to the Riemann problem is considered. When $u_L \leq u_R$, the solution corresponds to a rarefaction wave and the Godunov flux is,

$$F^{\mathrm{G}}_{i+\frac{1}{2}} = \begin{cases} \dfrac{u_L^2}{2}, & u_L \geq 0 \\ \dfrac{u_R^2}{2}, & u_R \leq 0 \\ 0, & u_L < 0 < u_R \end{cases} \tag{3}$$

When $u_L > u_R$, the solution corresponds to a shock wave and the Godunov flux is,

$$F^{\mathrm{G}}_{i+\frac{1}{2}} = \begin{cases} \dfrac{u_L^2}{2}, & u_L + u_R \geq 0 \\ \dfrac{u_R^2}{2}, & u_L + u_R < 0 \end{cases} \tag{4}$$

2.2 Flux Vector Splitting (FVS) Scheme

In the flux vector splitting (FVS) scheme, the flux is decomposed into left and right components,

$$f_L = \begin{cases} \frac{u_L^2}{2}, & u_L > 0, \\ 0, & u_L \leq 0, \end{cases} \qquad f_R = \begin{cases} \frac{u_R^2}{2}, & u_R < 0, \\ 0, & u_R \geq 0. \end{cases} \tag{5}$$

The numerical flux at the interface is therefore

$$F_{i+\frac{1}{2}}^{\text{FVS}} = f_L + f_R \tag{6}$$

2.3 Flux Difference Splitting (FDS) Scheme

In the flux difference splitting (FDS) scheme, the left and right physical fluxes are given by,

$$f_L = \frac{u_L^2}{2}, \qquad f_R = \frac{u_R^2}{2} \tag{7}$$

and the shock speed is given by,

$$s = \frac{u_L + u_R}{2} \tag{8}$$

The numerical flux is then,

$$F_{i+\frac{1}{2}}^{\text{FDS}} = \frac{1}{2}\left[f_L + f_R - |s|(u_R - u_L)\right] \tag{9}$$

3 Initialisation and Simulation Details

The governing equation is solved over a domain $x \in [-1, 1]$ with $N = 21$ and a timestep $\Delta t = 0.01$, for a total time of $0.5\,s$. The initial conditions are given below and are depicted in Fig. 1.

- Case 1:

$$u = \begin{cases} 1, & x < 0 \\ 0, & x > 0 \end{cases}$$

- Case 2:

$$u = \begin{cases} 0, & x < 0 \\ 1, & x > 0 \end{cases}$$

- Case 3:

$$u = \begin{cases} -1, & x < 0 \\ 1, & x > 0 \end{cases}$$

- Case 4:

$$u = \begin{cases} 1, & x \leq 0 \\ -1, & x > 0 \end{cases}$$

- Case 5:

$$u = \begin{cases} 1, & x \leq -0.5 \\ 0.5 - x, & -0.5 < x < 0.5 \\ 0, & x \geq 0.5 \end{cases}$$

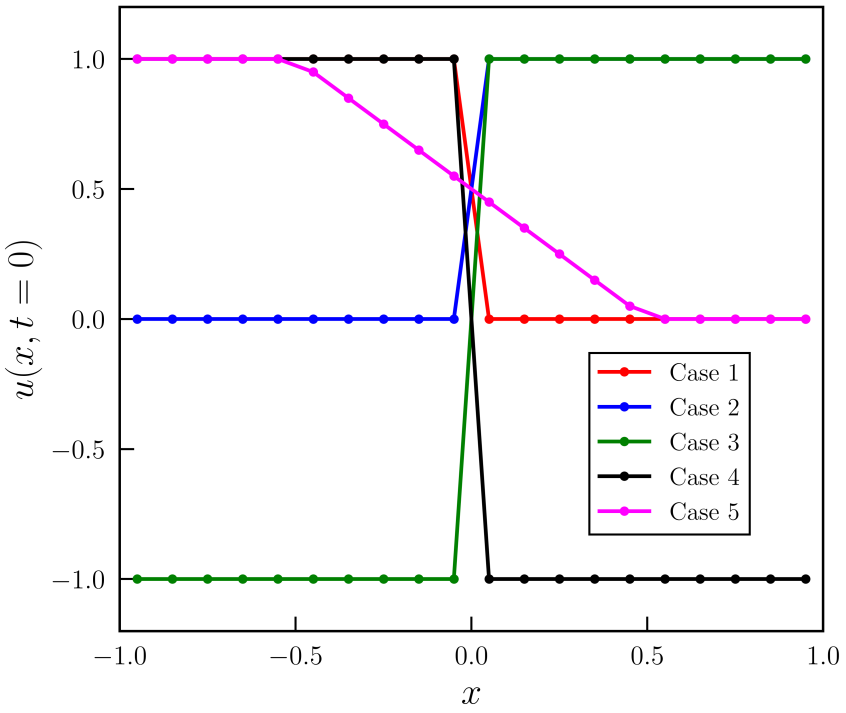


Figure 1: Initial Conditions

The flux residual represents the flux imbalance in a given cell and is given by,

$$R = f_i - f_{i-1} \tag{10}$$

4 Exact Solutions for the Inviscid Burgers Equation

4.1 Case 1: Moving Shock

The initial condition is,

$$u(x, 0) = \begin{cases} 1, & x \leq 0 \\ 0, & x > 0 \end{cases} \tag{11}$$

Since $u_L > u_R$, the solution develops a shock. The shock speed is obtained as,

$$s = \frac{f(u_L) - f(u_R)}{u_L - u_R} = \frac{\frac{1}{2} - 0}{1} = \frac{1}{2} \tag{12}$$

Hence, the shock is located at $x = st = t/2$, and the exact solution is,

$$u(x,t) = \begin{cases} 1, & x < \dfrac{t}{2} \\ 0, & x > \dfrac{t}{2} \end{cases} \tag{13}$$

Thus, the discontinuity propagates in the positive x-direction with a constant speed $s = 1/2$.

4.2 Case 2: Rarefaction

The initial condition is,

$$u(x,0) = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases} \tag{14}$$

Since $u_L < u_R$, the discontinuity evolves into a rarefaction wave. The characteristic speeds at the two states are,

$$f'(u_L) = 0, \qquad f'(u_R) = 1. \tag{15}$$

The exact solution is therefore,

$$u(x,t) = \begin{cases} 0, & x < 0 \\ \dfrac{x}{t}, & 0 \leq x \leq t \\ 1, & x > t \end{cases} \tag{16}$$

The rarefaction fan hence occupies the region $0 \leq x \leq t$.

4.3 Case 3: Symmetric Rarefaction

The initial condition is,

$$u(x,0) = \begin{cases} -1, & x \leq 0 \\ 1, & x > 0 \end{cases} \tag{17}$$

Since $u_L < u_R$, a rarefaction wave is formed. The characteristic speeds at the two edges of the fan are,

$$f'(u_L) = -1, \qquad f'(u_R) = 1 \tag{18}$$

The exact solution is,

$$u(x,t) = \begin{cases} -1, & x < -t \\ \dfrac{x}{t}, & -t \le x \le t, \\ 1, & x > t \end{cases} \tag{19}$$

Thus, the rarefaction fan expands symmetrically about $x = 0$.

4.4 Case 4: Stationary Shock

The initial condition is,

$$u(x,0) = \begin{cases} 1, & x \le 0 \\ -1, & x > 0 \end{cases} \tag{20}$$

Since $u_L > u_R$, a shock is formed. Its propagation speed is,

$$s = \frac{f(1) - f(-1)}{1 - (-1)} = \frac{\frac{1}{2} - \frac{1}{2}}{2} = 0 \tag{21}$$

Therefore, the shock remains stationary at $x = 0$, giving,

$$u(x,t) = \begin{cases} 1, & x < 0 \\ -1, & x > 0 \end{cases} \tag{22}$$

4.5 Case 5: Smooth Compression

The initial condition is,

$$u(x,0) = \begin{cases} 1, & x \le -\dfrac{1}{2} \\ \dfrac{1}{2} - x, & -\dfrac{1}{2} < x < \dfrac{1}{2} \\ 0, & x \ge \dfrac{1}{2} \end{cases} \tag{23}$$

For the linear portion of the initial condition,

$$u_0(\xi) = \frac{1}{2} - \xi \tag{24}$$

The characteristic equations for Burgers' equation are,

$$\frac{dx}{dt} = u, \qquad u = u_0(\xi) \tag{25}$$

where ξ denotes the initial position of a characteristic. Hence,

$$x = \xi + \left(\frac{1}{2} - \xi\right) t \tag{26}$$

Solving for ξ,

$$\xi = \frac{x - \frac{t}{2}}{1 - t} \tag{27}$$

Therefore, within the evolving linear region,

$$u(x,t) = \frac{1}{2} - \xi = \frac{\frac{1}{2} - x}{1 - t} \tag{28}$$

The characteristics originating at $\xi = -1/2$ and $\xi = 1/2$ define the boundaries of this region,

$$x_L = -\frac{1}{2} + t, \qquad x_R = \frac{1}{2} \tag{29}$$

Thus, before shock formation, the exact solution is

$$u(x,t) = \begin{cases} 1, & x < -\frac{1}{2} + t, \\ \frac{\frac{1}{2} - x}{1 - t}, & -\frac{1}{2} + t \leq x \leq \frac{1}{2}, \\ 0, & x > \frac{1}{2}. \end{cases} \tag{30}$$

The initial velocity gradient in the linear region is,

$$\frac{\partial u_0}{\partial x} = -1 \tag{31}$$

Consequently, the first characteristic intersection occurs at

$$t_s = -\frac{1}{\min\left(\partial u_0 / \partial x\right)} = 1 \tag{32}$$

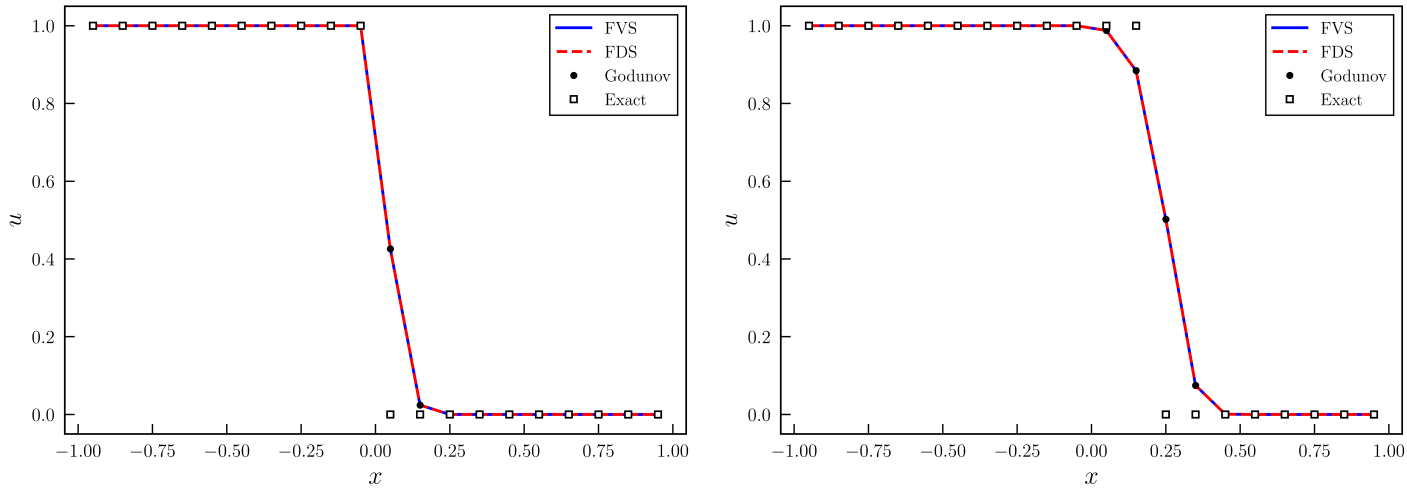
Since the numerical solutions considered here are evaluated only up to $t = 0.5$, the solution remains in the pre-shock regime, and the above analytical expression is valid throughout the comparison interval.

5 Results and Discussion

All the results are plotted at cell centers.

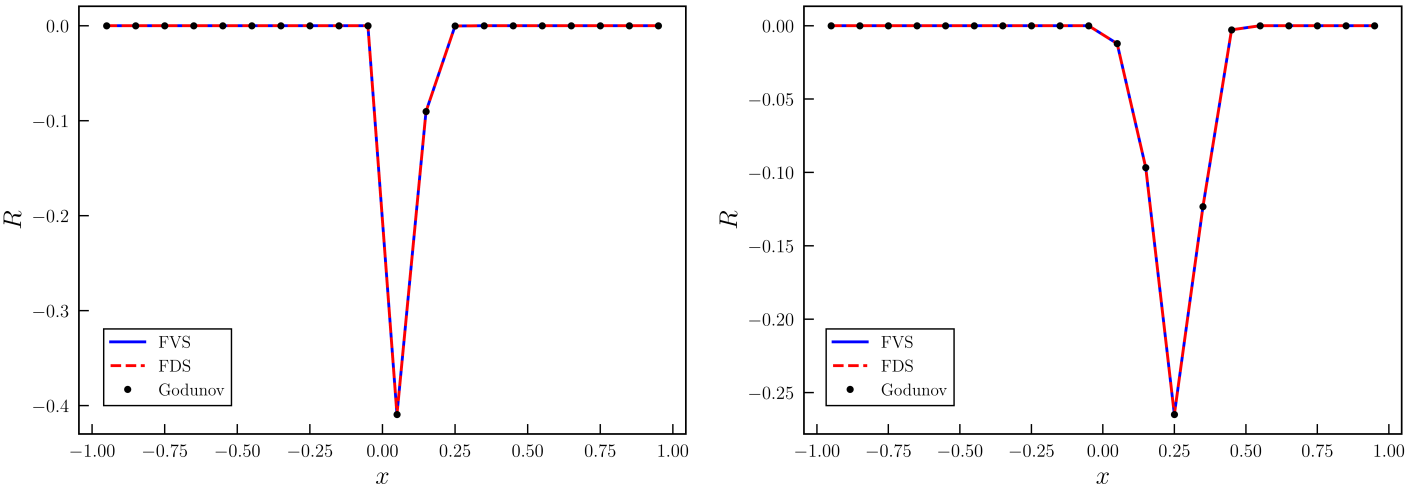
5.1 Case 1: Moving Shock

Case 1 corresponds to a rightwards moving shock, and this is supported by Fig. 2. The flux residuals are highest in the vicinity of the shock as observed from Fig. 3. The deviation from the exact solution can be attributed to the relatively coarse grid resolution. All the three schemes result in the same solution with no exceptions. This implies the fluxes computed via these schemes is consistent and identical.



(a) t = 0.1 s (b) t = 0.5 s

Figure 2: Solution corresponding to Case 1

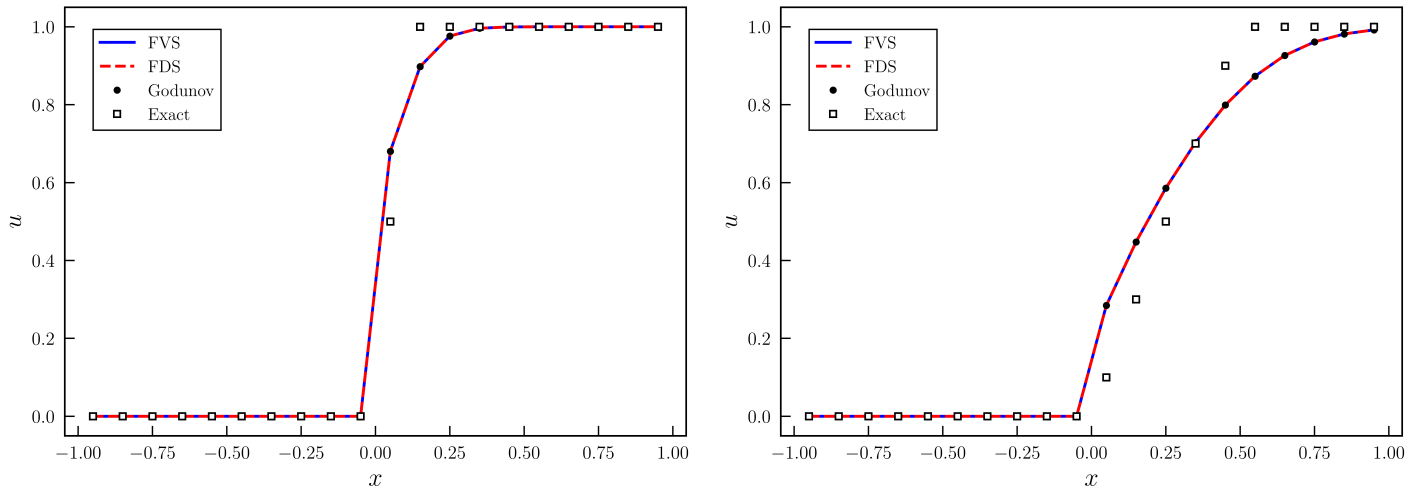


(a) t = 0.1 s (b) t = 0.5 s

Figure 3: Flux residuals corresponding to Case 1

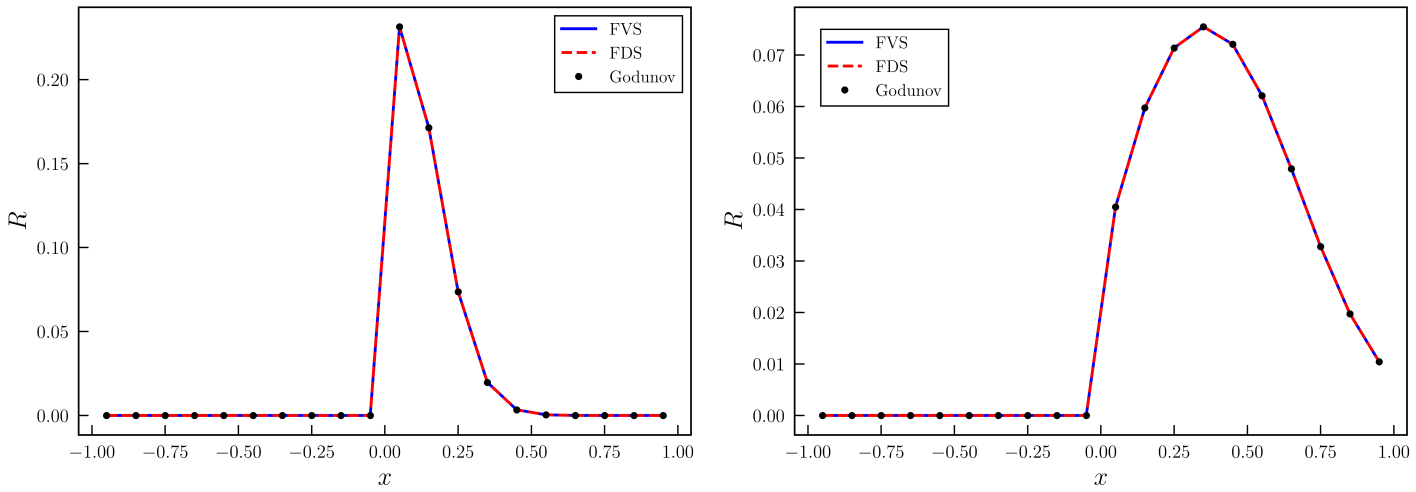
5.2 Case 2: Rarefaction

Case 2 corresponds to a rarefaction, and as evident from Fig. 4 this rarefaction spreads over time. Similar to Case 1, the deviation from the exact solution is quite significant and can be attributed to the coarse grid. Again the flux residuals vary significantly in the region of the rarefaction as supported by Fig. 5. Once again, all three schemes result in the same solution with no deviations, implying the fluxes computed are equivalent.



(a) t = 0.1 s (b) t = 0.5 s

Figure 4: Solution corresponding to Case 2

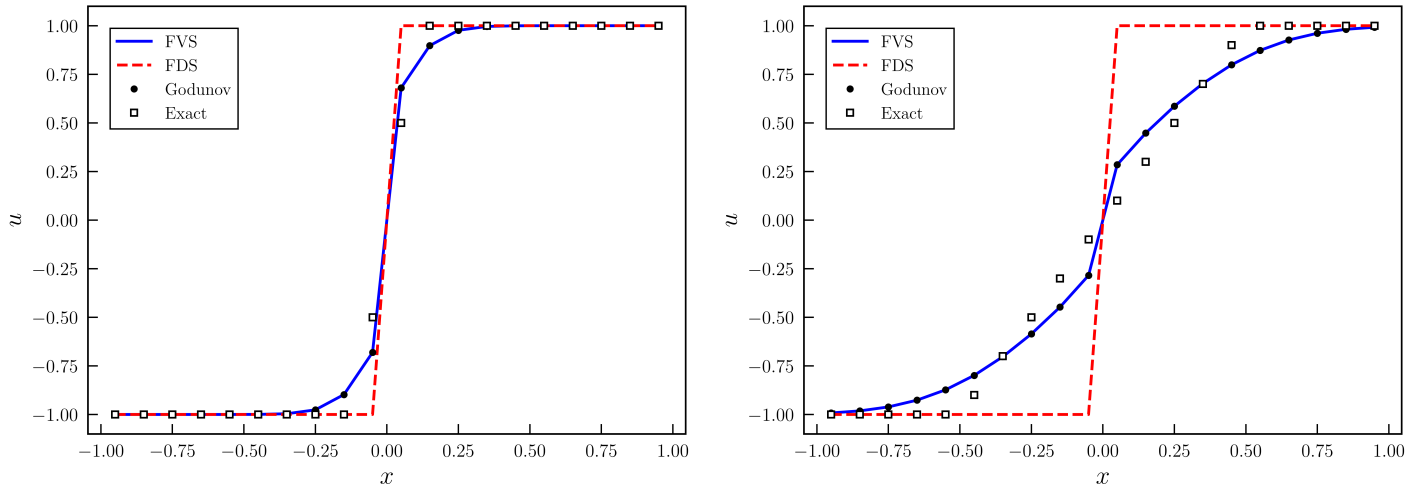


(a) t = 0.1 s (b) t = 0.5 s

Figure 5: Flux residuals corresponding to Case 2

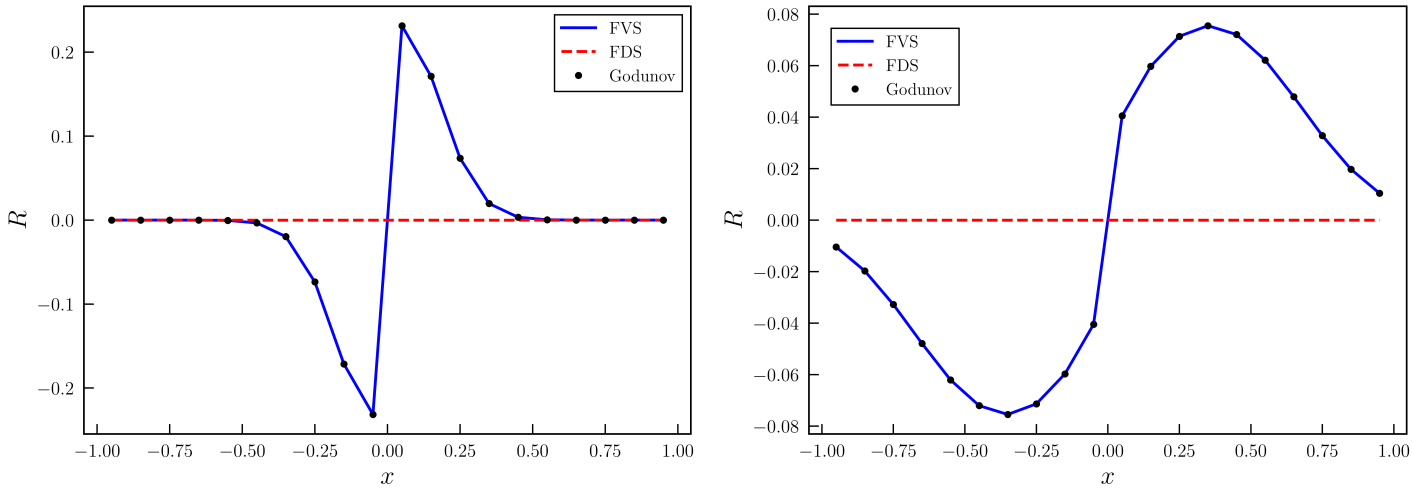
5.3 Case 3: Symmetric Rarefaction

Case 3 corresponds to a symmetric rarefaction about the origin, and this is clearly visible in Fig. 6. This is the first case where the solution predicted by different schemes diverge. Consequently, the fluxes computed by each scheme vary. The discrepancy in the solution with regards to the exact solution can be attributed to a relatively low resolution. The flux residuals in Fig. 7 also depict the flux increase in the direction of the rarefaction which is clearly not captured by the FDS scheme. The Godunov and FVS schemes are clearly superior in this case.



(a) t = 0.1 s (b) t = 0.5 s

Figure 6: Solution corresponding to Case 3

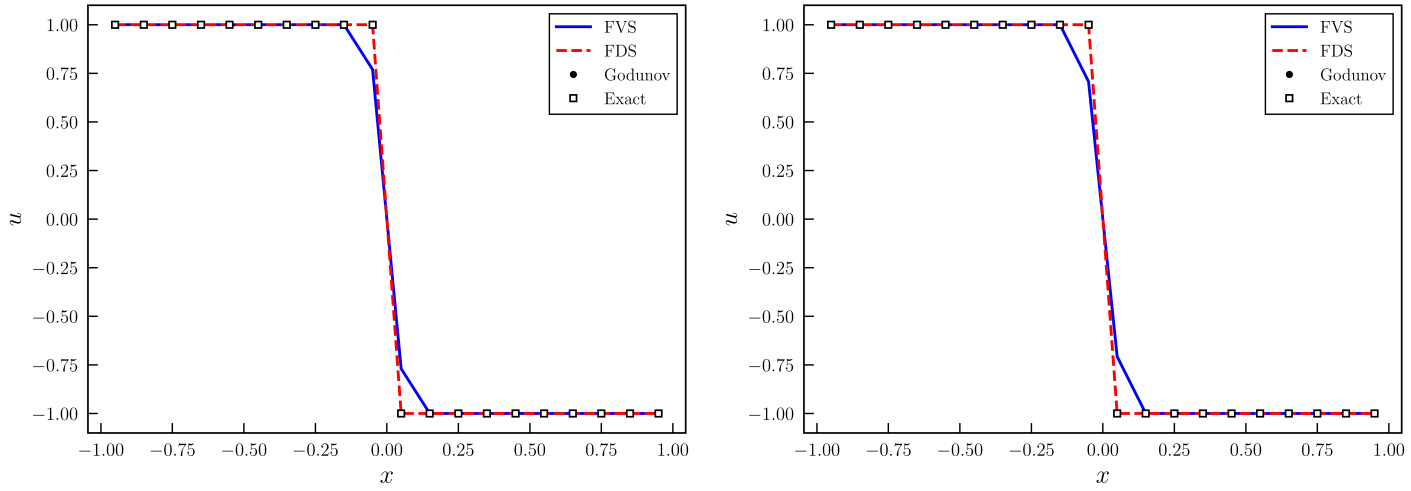


(a) t = 0.1 s (b) t = 0.5 s

Figure 7: Flux residuals corresponding to Case 3

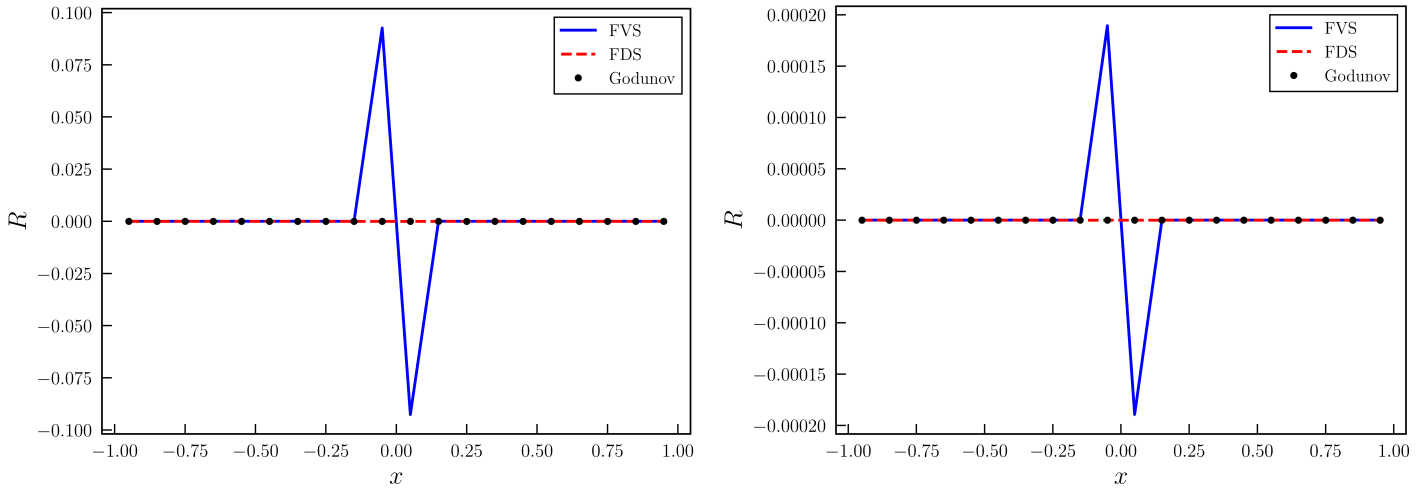
5.4 Case 4: Stationary Shock

Case 4 corresponds to a stationary shock, and as expected the discontinuity remains stationary as depicted in Fig. 8. In this case, however, the FDS and Godunov schemes are clearly superior, having captured the stationary discontinuity more accurately than the FVS scheme. Interestingly, this is in stark contrast to the previous case where it was exactly other way around between FVS and FDS. The FVS scheme seems to have 'smeared' the shock as evidenced by the flux residual plot in Fig. 7.



(a) t = 0.1 s (b) t = 0.5 s

Figure 8: Solution corresponding to Case 4

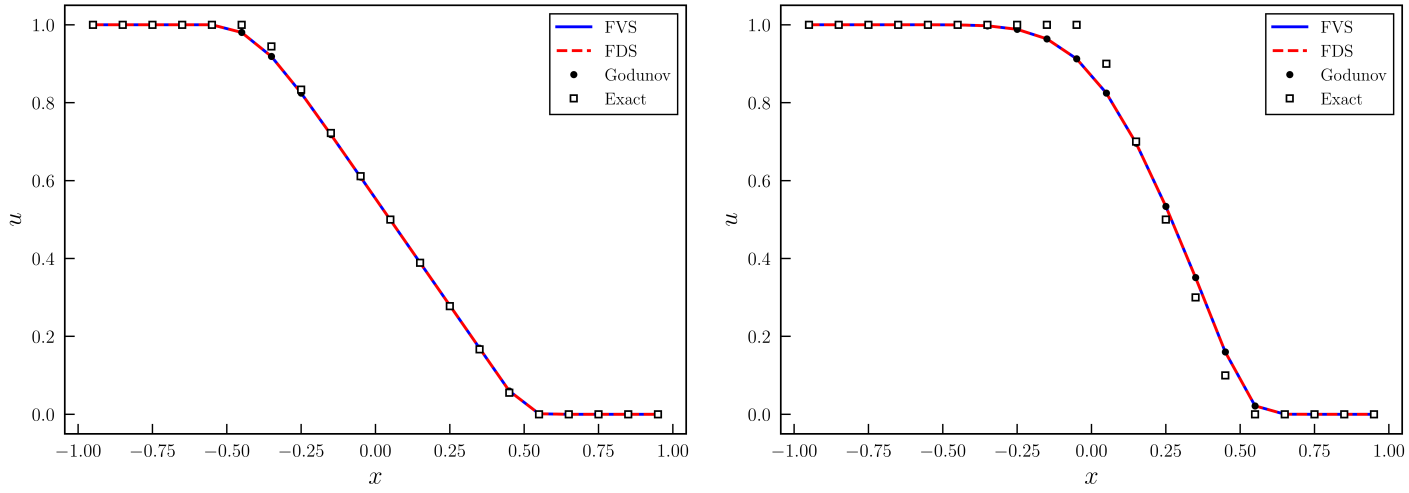


(a) t = 0.1 s (b) t = 0.5 s

Figure 9: Flux residuals corresponding to Case 4

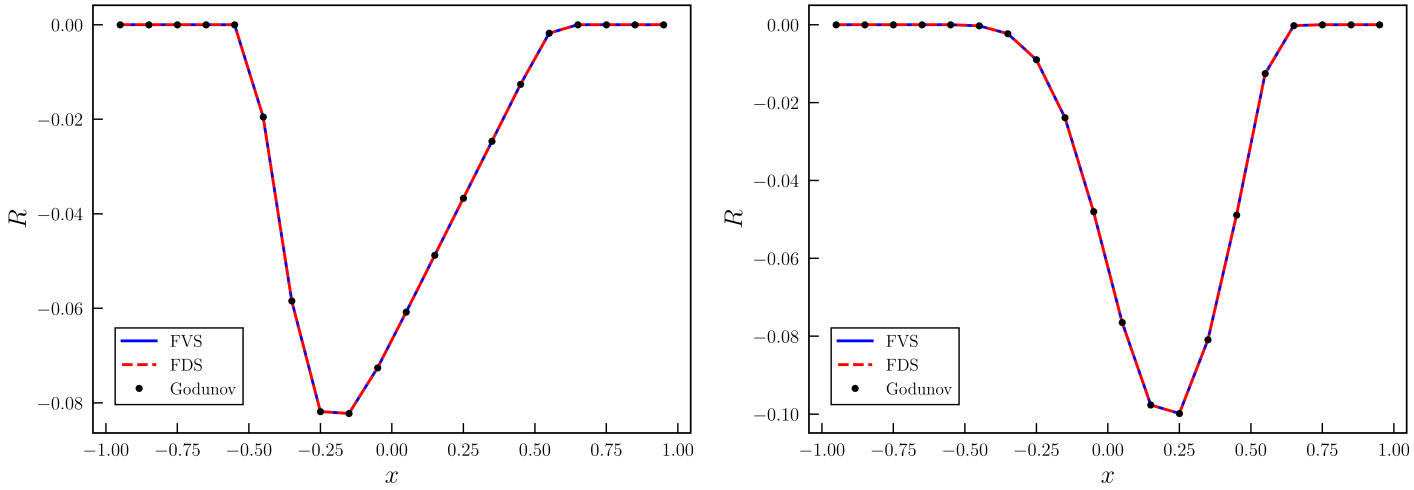
5.5 Case 5: Smooth Compression

Case 5 corresponds to a smooth compression, where the initial smooth initial condition compresses itself and forms an eventual discontinuity. This can be clearly identified from Fig. 10. In this case, all three schemes perform similarly, implying the same flux being is being computed across the board. The deviation from the exact solution can be attributed to resolution. The flux residuals in Fig. 11 are also consistent across the schemes.



(a) t = 0.1 s (b) t = 0.5 s

Figure 10: Solution corresponding to Case 5



(a) t = 0.1 s (b) t = 0.5 s

Figure 11: Flux residuals corresponding to Case 5

6 Conclusion

The inviscid Burgers' equation is solved numerically using the finite volume method. It is solved in an explicit fashion, with several flux construction schemes and initial conditions. The simulations were run in FORTRAN and post-processing is carried out using Python. The codes developed are exhibited in the appendix. Each of the schemes have their pitfalls with the Godunov flux construction scheme being the most consistent. Grid resolution is also extremely critical as evidenced by the results presented in this report. It is therefore prudent to run the simulations using a scheme dependent on the physical scenario and at an appropriate resolution.

7 References

[1] LeVeque, R., *Finite Volume Methods for Hyperbolic Problems*, Cambridge Texts in Applied Mathematics, Cambridge University Press, 2002.

A FORTRAN Implementation

A.1 Variable Declaration Module

```
1  module var_dec
2      implicit none
3
4      ! Define Double Precision
5      integer, parameter :: dp = kind(1.d0)
6
7      ! Initial Condition Case
8      integer :: icase, fcase
9
10     ! Define Domain and Grid Parameters
11     integer, parameter :: Lx = 2
12     integer, parameter :: N = 21
13     real(dp) :: xstart = -1.0_dp, xend = 1.0_dp
14
15     ! Define Time Parameters
16     real(dp) :: dx, dt = 0.01
17     real(dp) :: t, tend = 0.5_dp
18
19     ! Define Arrays
20     real(dp), allocatable :: x(:), xc(:)
```

```
21
22      real(dp), allocatable :: u(:), u_new(:)
23      real(dp), allocatable :: u_out_01(:,:), u_out_05(:,:)
24
25      real(dp), allocatable :: flux(:)
26      real(dp), allocatable :: res(:)
27      real(dp), allocatable :: r_out_01(:,:), r_out_05(:,:)
28
29      ! Helper Variables
30      integer(kind=8) :: i, iface
31  end module var_dec
```

A.2 Main Program

```
1   program burgers
2       use var_dec
3       implicit none
4
5       character(len=100) :: filename
6       integer :: nstep
7
8       ! Allocate Arrays
9       allocate(x(N), xc(N), u(N+1), u_new(N+1), flux(N), res(N))
10      allocate(u_out_01(N,3), u_out_05(N,3))
11      allocate(r_out_01(N,3), r_out_05(N,3))
12
13      ! Initialize Grid
14      dx = Lx / real(N - 1, dp)
15      do i = 1, N; x(i) = xstart + (i - 1) * dx; end do
16      do i = 2, N; xc(i) = 0.5 * (x(i) + x(i-1)); end do
17
18      do icase = 1, 5
19          do fcase = 1, 3
20
21              ! Assign Initial Condition
22              call ic(icase)
23
24              t = 0.0_dp
25              nstep = 0
26
27              do while (t < tend)
28                  nstep = nstep + 1
```

```
29
30              ! Boundary Condition
31              u(1) = u(2); u(N+1) = u(N)
32
33              ! Compute Flux
34              do iface = 1, N; i = iface; call cflux(u(i), u(i+1), fcase, flux(i)); end do
35
36              ! Compute Residual
37              do i = 2, N; res(i) = flux(i) - flux(i-1); end do
38
39              ! Write t = 0.1
40              if (nstep == 10) then
41                  do i = 2, N
42                      u_out_01(i,fcase) = u(i)
43                      r_out_01(i,fcase) = res(i)
44                  end do
45              end if
46
47              ! Write t = 0.5
48              if (nstep == 50) then
49                  do i = 2, N
50                      u_out_05(i,fcase) = u(i)
51                      r_out_05(i,fcase) = res(i)
52                  end do
53              end if
54
55              ! Update Solution
56              do i = 2, N; u_new(i) = u(i) - dt/dx * res(i); end do
57
58              ! Copy updated solution
59              do i = 2, N; u(i) = u_new(i); end do
60
61              ! Advance time
62              t = t + dt
63          end do
64      end do
65
66      ! Write results for the associated IC case
67      write(filename,'("results-",I0,".dat")') icase
68      open(unit=20, file=filename, status="replace", action="write")
69
```

```
        ! t = 0.1
        write(20,'(A)') '# t = 0.1'
        write(20,'(A)') '# x u_FVS R_FVS u_FDS R_FDS u_Godunov R_Godunov'

        do i = 2, N
            write(20,'(ES24.15E3,1X,ES24.15E3,1X,ES24.15E3,1X, &
                                ES24.15E3,1X,ES24.15E3,1X,ES24.15E3,1X,ES24.15E3)') &
                xc(i), &
                u_out_01(i,1), r_out_01(i,1), &
                u_out_01(i,2), r_out_01(i,2), &
                u_out_01(i,3), r_out_01(i,3)
        end do

        write(20,*)
        write(20,*)

        ! t = 0.5
        write(20,'(A)') '# t = 0.5'
        write(20,'(A)') '# x u_FVS R_FVS u_FDS R_FDS u_Godunov R_Godunov'

        do i = 2, N
            write(20,'(ES24.15E3,1X,ES24.15E3,1X,ES24.15E3,1X, &
                        ES24.15E3,1X,ES24.15E3,1X,ES24.15E3,1X,ES24.15E3)') &
                xc(i), &
                u_out_05(i,1), r_out_05(i,1), &
                u_out_05(i,2), r_out_05(i,2), &
                u_out_05(i,3), r_out_05(i,3)
        end do

        close(20)
    end do

    ! Deallocate Arrays
    deallocate(x, xc, u, u_new, u_out_01, u_out_05, r_out_01, r_out_05, flux, res)
end program burgers
```

A.3 Supporting Subroutines

```
subroutine ic(ic_case)
    use var_dec
    implicit none

```

```
    integer, intent(in) :: ic_case
    select case (ic_case)
        ! Case 1: u = 1 for x <= 0 and u = 0 for x > 0
        case (1)
            do i = 2, N
                if (xc(i) <= 0.0_dp) then; u(i) = 1.0_dp
                    else; u(i) = 0.0_dp
                end if
            end do

        ! Case 2: u = 0 for x <= 0 and u = 1 for x > 0
        case (2)
            do i = 2, N
                if (xc(i) <= 0.0_dp) then; u(i) = 0.0_dp
                    else; u(i) = 1.0_dp
                end if
            end do

        ! Case 3: u = -1 for x <= 0 and u = 1 for x > 0
        case (3)
            do i = 2, N
                if (xc(i) <= 0.0_dp) then; u(i) = -1.0_dp
                    else; u(i) = 1.0_dp
                end if
            end do

        ! Case 4: u = 1 for x <= 0 and u = -1 for x > 0
        case (4)
            do i = 2, N
                if (xc(i) <= 0.0_dp) then; u(i) = 1.0_dp
                    else; u(i) = -1.0_dp
                end if
            end do

        ! Case 5:
        ! u = 1 for x <= -0.5 and
        ! u = 0 for x >= 0.5
        ! linear for -0.5 < x < 0.5
        case (5)
            do i = 2, N
                if (xc(i) <= -0.5_dp) then; u(i) = 1.0_dp
```

```
                else if (xc(i) >= 0.5_dp) then; u(i) = 0.0_dp
                else; u(i) = 0.5_dp - xc(i)
            end if
        end do
    end select
end subroutine ic

subroutine cflux(uL, uR, scheme, F)
    use var_dec
    implicit none

    real(dp), intent(in) :: uL, uR
    integer, intent(in) :: scheme
    real(dp), intent(out) :: F

    real(dp) :: fL, fR
    real(dp) :: s

    select case (scheme)

    ! Flux Splitting
    case (1)

        if (uL > 0.0_dp) then; fL = 0.5_dp * uL**2
            else; fL = 0.0_dp
        end if

        if (uR < 0.0_dp) then; fR = 0.5_dp * uR**2
            else; fR = 0.0_dp
        end if

        F = fL + fR

    ! Flux Difference Splitting
    case (2)

        fL = 0.5_dp * uL**2
        fR = 0.5_dp * uR**2

        s = 0.5_dp * (uL + uR)
        F = 0.5_dp * (fL + fR - (abs(s) * (uR - uL)))
```

```
87
88      ! Godunov (Exact)
89      case (3)
90
91          if (uL <= uR) then
92
93              ! Rarefaction
94              if (uL >= 0.0_dp) then; F = 0.5_dp * uL**2
95                  else if (uR <= 0.0_dp) then; F = 0.5_dp * uR**2
96                  else; F = 0.0_dp ! Rarefaction crosses zero
97              end if
98
99          else
100
101             ! Shock
102             if (uL + uR >= 0.0_dp) then; F = 0.5_dp * uL**2
103                 else; F = 0.5_dp * uR**2
104             end if
105
106         end if
107
108     end select
109 end subroutine cflux
```

B Python Post-Processing

```
1  from pathlib import Path
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  # Plot style
6  plt.rcParams.update({
7      "text.usetex": True, "font.family": "serif",
8      "font.serif": ["Computer Modern Roman"],
9      "axes.labelsize": 18, "axes.titlesize": 16,
10     "xtick.labelsize": 13, "ytick.labelsize": 13,
11     "legend.fontsize": 12, "axes.linewidth": 1.2,
12     "xtick.direction": "in", "ytick.direction": "in",
13     "xtick.top": False, "ytick.right": False,
14     "xtick.major.size": 7, "ytick.major.size": 7,
15     "xtick.minor.size": 4, "ytick.minor.size": 4 })
```

```python

DATA, OUT = Path("."), Path("plots")

for d in ["ic", "sol/t01", "sol/t05", "res/t01", "res/t05"]:
    (OUT/d).mkdir(parents=True, exist_ok=True)

def format_axes(ax, ylabel):
    ax.set(xlabel=r"$x$", ylabel=ylabel)
    ax.grid(False)
    ax.tick_params(which="major", direction="in", length=7, width=1, pad=7)
    ax.tick_params(which="minor", direction="in", length=4, width=0.8)
    for s in ax.spines.values():
        s.set_linewidth(1.2)
        s.set_color("black")

def finish(fig, ax, filename, legend_specific=False):
    if legend_specific:
        ax.legend(
            loc="center left", bbox_to_anchor=(0.65, 0.3),
            frameon=True, fancybox=False, framealpha=1,
            edgecolor="black",
        )
        fig.tight_layout(rect=[0, 0, 0.84, 1])
    else:
        leg = ax.legend(
            loc="best", frameon=True, fancybox=False,
            framealpha=1, edgecolor="black",
        )
        fig.canvas.draw()
        bbox = leg.get_window_extent(fig.canvas.get_renderer())
        bbox = bbox.transformed(ax.transAxes.inverted())

        if bbox.x0 < 0.5:
            dy = -0.01 if bbox.y0 > 0.5 else 0.05
            bbox.x0 += 0.02
            bbox.x1 += 0.02
            bbox.y0 += dy
            bbox.y1 += dy
            leg.set_bbox_to_anchor(bbox, transform=ax.transAxes)

        fig.tight_layout()
```

```
57
58      fig.savefig(filename, dpi=600, bbox_inches="tight")
59      plt.close(fig)
60
61  def read_results(filename):
62      blocks, block = [], []
63
64      with open(filename) as f:
65          for line in f:
66              line = line.strip()
67
68              if not line:
69                  if block:
70                      blocks.append(np.array(block))
71                      block = []
72                  continue
73
74              if not line.startswith("#") and len(v := line.split()) == 7:
75                  block.append([float(x) for x in v])
76
77      if block:
78          blocks.append(np.array(block))
79
80      return blocks[0], blocks[1]
81
82  def initial_conditions():
83      x = np.linspace(-1, 1, 21)
84      xc = (x[1:] + x[:-1]) / 2
85
86      u = {
87          1: np.where(xc <= 0, 1, 0),
88          2: np.where(xc <= 0, 0, 1),
89          3: np.where(xc <= 0, -1, 1),
90          4: np.where(xc <= 0, 1, -1),
91          5: np.where(xc <= -0.5, 1, np.where(xc >= 0.5, 0, 0.5 - xc))}
92
93      return xc, u
94
95  def plot_initial_conditions():
96      x, u = initial_conditions()
97      colors = ["red", "blue", "green", "black", "magenta"]
```

```
98
99      fig, ax = plt.subplots(figsize=(7, 5))
100
101     for ic, color in enumerate(colors, 1):
102         ax.plot(
103             x, u[ic], color=color, linewidth=1.8,
104             marker="o", markersize=3.5, label=f"Case {ic}",
105         )
106
107     format_axes(ax, r"$u(x, t = 0)$")
108     ax.set(
109         xlim=(-1, 1), ylim=(-1.2, 1.2),
110         xticks=np.arange(-1, 1.1, 0.5),
111         yticks=np.arange(-1, 1.1, 0.5),
112     )
113
114     finish(fig, ax, OUT/("ic/ic.png"), True)
115
116 def exact_solution(ic, x, t):
117     x = np.asarray(x)
118
119     if ic == 1:
120         return np.where(x < t/2, 1.0, 0.0)
121
122     if ic == 2:
123         return np.where(x < 0, 0,
124                 np.where(x > t, 1, x/t))
125
126     if ic == 3:
127         return np.where(x < -t, -1,
128                 np.where(x > t, 1, x/t))
129
130     if ic == 4:
131         return np.where(x < 0, 1.0, -1.0)
132
133     if ic == 5:
134         xl = -0.5 + t
135         return np.where(x < xl, 1, np.where(x > 0.5, 0, (0.5 - x)/(1 - t)))
136
137
138 def plot_result(ic, time, data, residual=False):
```

```
    x = data[:, 0]
    cols = [2, 4, 6] if residual else [1, 3, 5]
    ylabel = r"$R$" if residual else r"$u$"
    directory = OUT/("res" if residual else "sol")/f"t{int(10*time):02d}"

    fig, ax = plt.subplots(figsize=(7, 5))

    ax.plot(x, data[:, cols[0]], "b-", linewidth=2, label="FVS")
    ax.plot(x, data[:, cols[1]], "r--", linewidth=2, label="FDS")
    ax.plot(x, data[:, cols[2]], "ko", markersize=4, label="Godunov")

    if not residual:
        ax.plot(
            x, exact_solution(ic, x, time), "ks",
            markersize=4.5,
            markerfacecolor="white",
            markeredgewidth=1.2,
            label="Exact",
            zorder=2,
        )

    format_axes(ax, ylabel)
    finish(fig, ax, directory/f"IC{ic}.png")

plot_initial_conditions()

for ic in range(1, 6):
    filename = DATA/f"results-{ic}.dat"

    print(f"Processing {filename}")
    t01, t05 = read_results(filename)

    for time, data in [(0.1, t01), (0.5, t05)]:
        plot_result(ic, time, data)
        plot_result(ic, time, data, True)

print("\nDone.")
```